

City Mode

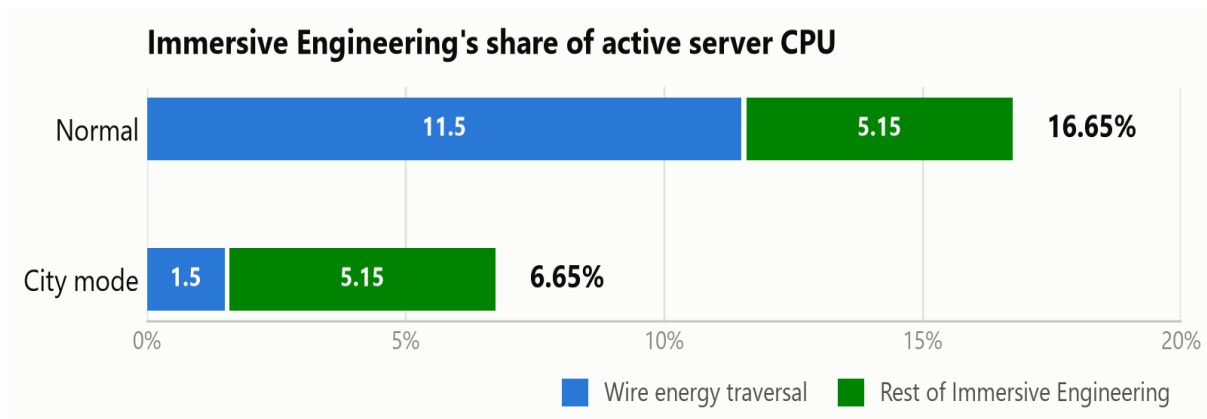
A config-gated mode for Immersive Engineering 1.12.2 that keeps the look of the mod and removes the simulation behind it.

City mode trades simulation detail for server tick time. It is aimed at city and roleplay packs where the mod's machinery is set dressing rather than an engineering puzzle: you keep the entire look of the build and give up the physics behind it. It covers four subsystems.

Subsystem	What is simplified
Wires	One lossless push per connector instead of loss, distance weighting, proportional split, a double simulate/real pass and a network-wide broadcast.
Floodlights	Beams re-traced only when the light switches or a neighbour changes, never on a timer; light-block count capped per lamp.
Generators	Fuel becomes cosmetic — a presence check and a token sip instead of a per-tick burn rate and tank drain.
Machines	Idle multiblocks stop re-scanning the recipe list every tick; the scan interval widens.

It is **off by default**, and off means byte-identical to stock. `cityMode` is the master switch; `cityModeWires`, `cityModeFloodlights`, `cityModeGenerators` and `cityModeMachines` each default to on, so the master alone enables everything while any one subsystem can be declined. Switching the master off is always sufficient to restore stock behaviour, and nothing here touches saved data.

Modelled performance — the wire subsystem

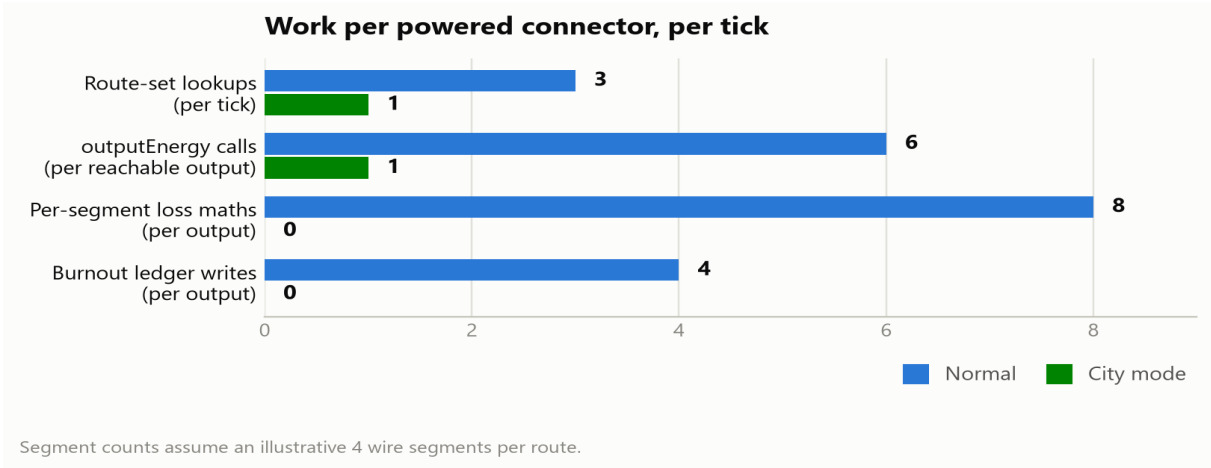


These are modelled figures, not measurements, and they cover the wire subsystem only. They apply the operation counts below to the one measured baseline this fork has — a live spark profile of the production server, where Immersive Engineering was the single most expensive individual mod at 16.65% of active CPU, roughly 11.5 points of which was the wire traversal alone. City mode itself has not yet been profiled under live load. Treat the shape as sound and the exact percentages as an estimate.

The floodlight, machine and generator work all lives inside the undifferentiated 5.15% "rest of IE" slice and is deliberately **not** modelled here, because its value depends entirely on what you built. Floodlights scale with how many lamps you have and how obstructed their beams are — for a lit city that could plausibly exceed the wire saving, and it is the one to measure first. Machines scale with how many multiblocks sit idle holding unusable input. Generators are negligible by design; they are in

for the gameplay semantics, not the CPU. Toggle the sub-flags individually to separate their contributions.

Where the saving comes from



The reduction is structural rather than incremental. Normal mode fetches the route set three times per tick and runs the distribution twice — a simulate pass to discover demand, then a real pass that sorts every output into a TreeMap by loss rate, gives each a proportional share, applies per-segment attenuation and records throughput for wire burnout — followed by a third full walk of the network to advertise available energy. City mode does one lookup, one real push per output, and stops.

What changes, in gameplay terms

	Normal	City mode
Wire loss	2.5–5% per 16 blocks, worse when lightly loaded	none
Voltage tiers	throttle to the weakest wire on the path	cosmetic only
Supply < demand	proportional to demand, nearest first	greedy, arbitrary order
Wire burnout	wire destroyed above its rate	cannot happen
Wire shock damage	sourced from the whole network	sourced from the local connector
Breaker switches	cut the network	unchanged
Energy meters	read loss-attenuated throughput	read full throughput
Floodlight beams	re-traced every 512 ticks	only on switch / neighbour change
Floodlight light count	uncapped	capped at 64 per lamp
Generator fuel burn	per-tick rate from the fluid	1 mB every 20 ticks, cosmetic
Generator load gate	only runs under load	unchanged — deliberately kept
Idle machine recipe scan	every tick	every 32 ticks

	Normal	City mode
Machine speed / power needs	—	unchanged

Three consequences worth knowing

Floodlights stop noticing distant obstructions

A floodlight is usually the most expensive block in a city build. Every 512 ticks each one re-traces thirteen beams, queues a light block roughly every third block along each, and recalculates block lighting — and every light it places is an individually ticking tile entity, uncapped, so one unobstructed lamp can own well over a hundred. City mode rebuilds only when the light switches or a neighbouring block changes. The cost is that a wall built across a beam further out is not noticed until something else triggers a rebuild, leaving the lights beyond it floating until the lamp is toggled.

Wire burnout is disabled

Overload destruction works by the normal transfer path recording per-connection throughput into a ledger that a world-tick handler then acts on. That path is the ledger's only writer, so in city mode it stays empty and no wire can ever burn out. Combined with the fact that city mode never clamps to the cable's rate, a copper wire will carry a full HV connector's 4096 FE/t forever with no consequence. That is intended for a city pack, but it is a rules change rather than an optimisation.

Wire shock damage becomes local

Damage is computed from a per-tick list of energy sources on each connectable, which the skipped network broadcast used to populate. In city mode that list holds only the connector's own energy, so damage is generally lower and a wire span running between two relays will not shock at all, because relays never hold energy. Set `enableWireDamage = false` to turn it off properly.

Does a diesel generator still need fuel?

Yes. A diesel generator gates on two conditions: something must actually accept power, and there must be fuel in the tank. With no consumers, connector buffers saturate, the generator's simulated insert returns zero, the fan spins down and no fuel burns — in both modes. What city mode makes cosmetic is the *rate*: instead of deriving a per-tick burn rate from the fluid and draining every tick, it sips 1 mB every 20 ticks, so a full tank lasts about six and a half hours of runtime.

The load gate is kept deliberately. Only running when something wants power is a *performance* feature, not a realism one — it is what makes an idle generator free. Removing it would make city mode slower, not faster, because generators would push energy at saturated connectors every tick only for it to be discarded.

City mode also raises the yield per unit of fuel, because the 4096 FE/t leaving the generator arrives undiminished instead of being attenuated by every wire segment on the way. More useful power per bucket of biodiesel, but never power from nothing — only what receivers actually accepted is deducted from the connector.

Block-by-block summary

Block	Role	Affected by city mode
Diesel Generator	4096 FE/t, burns fuel only under load	Yes — fuel burn becomes a token sip; load gate kept
Thermoelectric / Dynamo	push to adjacent blocks only	No — already cached or event-driven
Windmill / Water Wheel	drive the dynamo, produce no FE	No — already cached and throttled
Floodlight	13 beams of ticking light blocks	Yes — no timed re-scan, 64-light cap
Multiblock machines	Arc Furnace, Squeezer, Fermenter, Mixer, Refinery	Yes — idle recipe scan throttled to 32 ticks
Capacitor LV/MV/HV	buffer; 256 / 1024 / 4096 FE/t	No
Creative Capacitor	infinite source, all six sides	No — works as an infinite source in both modes
Lightning Rod	16,000,000 FE on a strike	No
Connector LV/MV/HV	the only blocks moving energy over wires	Yes — push path replaced; its own rate caps still apply
Relay LV/MV/HV	routing waypoint, never holds energy	No — tick body never runs either way
Transformer	tier adapter; holds no energy	Yes — tier throttling disappears
Breaker Switch	interrupts the network	No — still cuts
Energy Meter	passive throughput accumulator	Works, and reads the full unattenuated amount
Machine power intake	receive by push into their own buffer	No — their own intake caps still apply
Direct generator → machine	no wires involved	No — entirely unaffected

Full technical detail, including the annotated transfer routine, wire-type tables and the testing checklist, is in `docs/CITY_MODE.md`.